EXPT NO:8	A PYTHON PROGRAM TO IMPLEMENT THE GRADIENT DESCENT METHOD
DATE:	

AIM:

To iteratively adjust the parameters of a model in order to minimize the error between the predicted and actual values by using the Stochastic Gradient Descent (SGD) algorithm. SGD updates the model parameters based on each training example, allowing for faster convergence and handling larger datasets efficiently compared to batch gradient descent.

PROCEDURE:

1.INITIALIZATION:

- We initialize the weights to zeros.
- X is the feature matrix, and y is the target vector.
- Learning rate controls the step size of each update.
- Epochs are the number of iterations over the entire dataset.

2.GRADIENT COMPUTATION:

- For each sample in the dataset, we compute the gradient of the loss with respect to the weights.
- The gradient is calculated as (prediction - target) * feature.

3.WEIGHTS UPDATE:

- We update the weights by moving them in the opposite direction of the gradient, scaled by the learning rate.

PROBLEM 1: TO APPLY STOCHASTIC GRADIENT DESCENT (SGD) TO A SIMPLE LINEAR REGRESSION MODEL & FIND THE OPTIMAL WEIGHTS.

X	2.00	1.00	...	2.00
y	10.04	6.23	...	12.89

CODE

```
import numpy as np

def stochastic_gradient_descent(X, y, learning_rate=0.01, epochs=1000): n_samples, n_features =
X.shape weights = np.zeros(n_features)

for epoch in range(epochs):
    for i in range(n_samples):
        gradient = (np.dot(X[i], weights) - y[i]) * X[i]
        weights -= learning_rate * gradient

return weights

if name == "main": np.random.seed(0)

X = 2 * np.random.rand(100, 1)
y = 4 + 3 * X + np.random.randn(100, 1)
X_b = np.c_[np.ones((100, 1)), X]
y = y.ravel()
learning_rate = 0.01
epochs = 100
weights = stochastic_gradient_descent(X_b, y, learning_rate, epochs)
print("Weights:", weights)
```

OUTPUT

Weights: [4.16464189 2.90371342]

PROBLEM 2: TO APPLY STOCHASTIC GRADIENT DESCENT (SGD) TO A SIMPLE LINEAR REGRESSION MODEL & FIND THE OPTIMAL WEIGHTS.

Use **Stochastic Gradient Descent** to estimate parameters of the linear model:

$y=2+5x+noise$ $y=2+5x+\text{noise}$

CODE

```
import numpy as np

def stochastic_gradient_descent(X, y, learning_rate=0.01, epochs=1000): n_samples, n_features =
X.shape weights = np.zeros(n_features)

for epoch in range(epochs):
    for i in range(n_samples):
        gradient = (np.dot(X[i], weights) - y[i]) * X[i]
        weights -= learning_rate * gradient
```

```
return weights
 if name == "main": np.random.seed(1)

X = 3 * np.random.rand(100, 1)
y = 2 + 5 * X + np.random.randn(100, 1)
X_b = np.c_[np.ones((100, 1)), X]
y = y.ravel()
weights = stochastic_gradient_descent(X_b, y, 0.01, 1000)
print("Weights:", weights)
```

OUTPUT

Weights: [2.10 4.95]

RESULT

Thus, solving any Gradient Descent Method using the Stochastic Gradient Descent Algorithm in python has been implemented and executed successfully.